

STM32 嵌入式入门

第 1 讲 GPIO：点亮第一盏灯

邝姿霖

浙江大学学生机器人协会·教学部

本讲目标

学完这 10 分钟，你应该能够：

- ① 说出单片机是什么、STM32 和 Arduino 是什么关系
- ② 看懂 STM32F103C8T6 这串型号
- ③ 知道 GPIO 输出高低电平是怎么控制 LED 的
- ④ 用 CubeMX + HAL 库写出让 LED 闪烁的程序

本讲用到的硬件

STM32F103C8T6 最小系统板（“蓝色小板”）+ ST-Link V2 下载器

单片机是什么？

单片机 = 把一台电脑做进一颗芯片

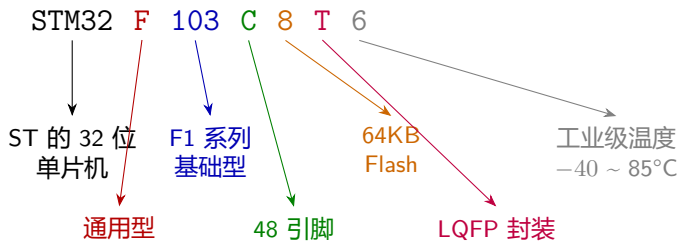
- CPU：执行指令
- Flash：存程序（断电不丢）
- RAM：存变量
- **外设**：GPIO、定时器、串口、ADC.....
负责和外部世界打交道

Arduino UNO 上的 ATmega328P 也是单片机；
STM32 是 ST 公司基于 ARM Cortex-M 内核
的一系列单片机。

	UNO	F103C8T6
内核	8 位 AVR	32 位 Cortex-M3
主频	16 MHz	72 MHz
Flash	32 KB	64 KB
RAM	2 KB	20 KB
I/O 电平	5V	3.3V

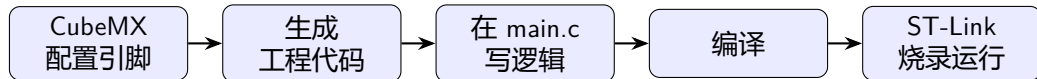
性能更强、外设更多，
是机器人比赛中最常见的主控

读懂型号：STM32F103C8T6



以后拿到任何一个 STM32，先看型号就知道它大概有多少引脚、多大存储。

开发流程



- 工具: **STM32CubeIDE** (免费, CubeMX 已集成在里面)
- **CubeMX**: 用图形界面点选 “这个引脚做什么”, 自动生成初始化代码
- **HAL 库**: ST 官方函数库, 类似 Arduino 的 `digitalWrite()`

GPIO：通用输入输出口

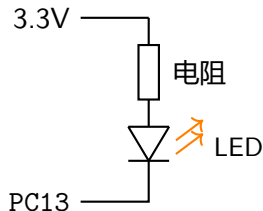
- GPIO = General Purpose Input/Output
- 引脚按组命名：PA0-PA15、PB0-PB15、PC13 等
- 配置成**输出**时，程序可以让它变成：
 - 高电平：3.3V（逻辑 1）
 - 低电平：0V（逻辑 0）
- 配置成**输入**时，可以读出外部是高还是低（下一讲：按键）

两种输出模式

推挽：能主动输出高和低，**驱动 LED 用这个**

开漏：只能主动拉低，高电平要靠外部上拉电阻（用于 I²C 等总线）

板载 LED 是怎么接的？



- LED 的正极接 3.3V，负极经 PC13 引出
- PC13 输出**低电平**：两端有电压差 → 电流流过 → **灯亮**
- PC13 输出高电平：两端都是 3.3V → 没有电流 → 灯灭

记住

蓝色小板的板载 LED 是**低电平点亮**，和直觉相反。写代码前先看原理图！

在 CubeMX 里配置

- ① 新建工程，芯片选 STM32F103C8Tx
- ② **System Core** → **SYS** → **Debug** 选 Serial Wire
(不选的话，第二次下载时 ST-Link 会连不上芯片)
- ③ **System Core** → **RCC** → **HSE** 选 Crystal/Ceramic Resonator，时钟树里把主频设为 72 MHz
- ④ 在芯片图上点击 PC13，选 GPIO_Output
- ⑤ 右侧 GPIO 设置里：模式 Output Push Pull，给它起个名字 LED
- ⑥ 保存 (Ctrl+S)，自动生成代码

写代码：让 LED 闪烁

```
int main(void)
{
    HAL_Init(); // HAL 库初始化
    SystemClock_Config(); // 时钟配置 (CubeMX 生成)
    MX_GPIO_Init(); // GPIO 初始化 (CubeMX 生成)

    while (1)
    {
        /* USER CODE BEGIN 3 */
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET); // 低电平：亮
        HAL_Delay(500); // 等待 500 ms
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET); // 高电平：灭
        HAL_Delay(500);
        /* USER CODE END 3 */
    }
}
```

- 也可以用一行 `HAL_GPIO_TogglePin(LED_GPIO_Port, LED_Pin);` 翻转电平
- `LED_GPIO_Port`、`LED_Pin` 是 CubeMX 根据你起的名字自动生成的宏

HAL 函数背后发生了什么？

HAL_GPIO_WritePin 最终做的事情只有一行：写 BSRR 寄存器

```
GPIOC->BSRR = (1 << 13); // 低 16 位写 1: PC13 置高（灯灭）  
GPIOC->BSRR = (1 << (13 + 16)); // 高 16 位写 1: PC13 置低（灯亮）
```

- 寄存器就是芯片里一个个有地址的“开关”，往里写数，硬件就照做
- HAL 库帮我们吧“写哪个地址、哪一位”包装成了好读的函数
- 好处：可读、可移植；代价：多了几层函数调用，速度稍慢

调试时可以在 CubeIDE 的 SFRs 窗口里实时看到 GPIOC 寄存器的值变化。

新手最常踩的坑

代码写了，重新生成后不见了

自己的代码必须写在 `/* USER CODE BEGIN */` 和 `/* USER CODE END */` 之间，否则 CubeMX 重新生成时会被覆盖。

第二次下载失败：No target connected

忘了把 SYS → Debug 设为 Serial Wire。按住板上 RESET 键点下载、看到连接后松开即可恢复。

程序没问题，灯就是不亮 / 一直亮

检查 LED 是高电平亮还是低电平亮；检查引脚名是否和原理图一致。

三句话回顾

- ① STM32 是 32 位单片机，比 Arduino 更强，I/O 是 3.3V
- ② GPIO 输出就是让引脚变成 3.3V 或 0V；蓝色小板的 LED 低电平点亮
- ③ 开发流程：CubeMX 配置 → 生成代码 → USER CODE 区写逻辑 → 编译烧录

课后练习（任选其一，下节课抽查）：

- 在面包板上接 4 个 LED，实现**流水灯**
- 用板载 LED 闪出摩尔斯电码 **SOS**（短短短长长短短短）

下一讲预告：GPIO 输入——用按键控制 LED，以及为什么需要“消抖”。

谢谢！

课件与示例工程：协会 GitHub 仓库 `slides/`